

Cours 1 : Introduction à l'Algorithmique

Baligh Mnassri

ECE Paris

Plan du cours

- 1 Préambule : Informations sur la matière
- 2 Définitions
- 3 Cahier des charges (CDC)
- 4 Étapes de développement d'un programme
- 5 Analyse du CDC
- 6 Algorithmique
- 7 Logigramme
- 8 Exercices
- 9 Tester son programme

Préambule : Informations sur la matière



L'informatique à l'ECE

- ✓ Omniprésente dans l'école, mais aussi en entreprise
- ✓ Pas de prérequis, nous partons de zéro.
- ✓ Ne ratez pas le départ.
- ✓ Veillez à comprendre au fur et à mesure.
- ✓ Faites-vous des fiches de révision, notez les points difficiles.
- ✓ Ne restez pas bloqué.e en TD/TP, essayez des choses, votre ordinateur ne va pas exploser.
- ✓ Demandez de l'aide.

 Les cours

Préparer et suivre efficacement

- ✓ Les cours sont disponibles à l'avance. **Lisez-les** et préparez vos questions avant chaque séance.
- ✓ Prenez en notes ce qui est ajouté à l'oral. **Synthétisez.**

 Matériel

Ce que vous devez apporter

- ✓ **Ordinateur obligatoire** à chaque séance (CI, TD et TP), avec un **environnement de développement fonctionnel**.
- ✓ Souris conseillée pour plus de confort.

 Communication Restez connectés

- ✓ **E-mail** : ajoutez votre compte e-mail ECE sur votre smartphone pour ne rien manquer.
- ✓ Synchronisez également votre emploi du temps !



Programme annuel

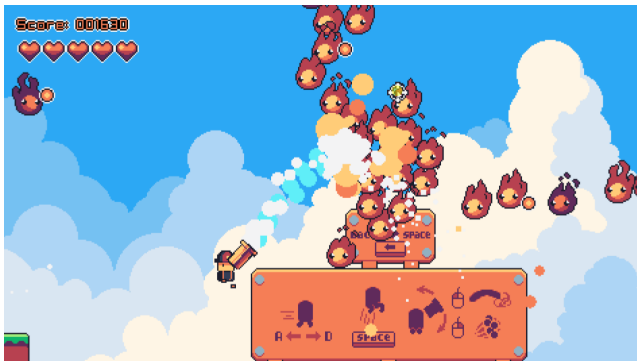
Semestre 1

- ✓ Algorithmique
- ✓ Langage C (en mode console)

Semestre 2

- ✓ Algorithmique avancée (récursivité...)
- ✓ Structures de données avancées :
 - Listes chaînées
 - Arbres binaires de recherche
 - Piles
 - Files
- ✓ Interfaces graphiques avec **Raylib**

🎮 Ce que vous serez capable de faire dans à peine 7 mois* !



**Si vous travaillez régulièrement et n'accumulez aucun retard.*



Évaluations par semestre

- ☐ 1 partiel
- ☐ 1 projet en groupe
- ☐ 3 notes de suivi, ou plus :
 - Interrogations (potentiellement surprises 😊)
 - Devoirs à rendre / évaluation de TPs
 - Participation / comportement

Définitions

☰ Définitions

- ✓ **Informatique** : science du traitement automatique de l'information.
- ✓ **Algorithme** : suite logique et ordonnée d'instructions à suivre et d'opérations à effectuer sur des données pour obtenir un résultat.
- ✓ **Programme informatique** : ensemble d'instructions et d'opérations destinées à être exécutées par un ordinateur.
- ✓ Un **programme** est la traduction d'un ou plusieurs algorithmes.
- ✓ **Programme (code) source** : la suite d'instructions écrite dans un langage de programmation (C, Python, C++, Java...).
- ✓ **Langage C** : inventé au début des années 1970. L'un des langages les plus utilisés, encore aujourd'hui. Sa syntaxe a inspiré de nombreux autres langages (C++, C#, Java, JavaScript, PHP...).
- ✓ C'est un **langage compilé** : le code source est traduit en langage machine (binaire) par un **compilateur**, avant d'être exécuté.
- ✓ **Programmation structurée en C** : les programmes sont décomposés en sous-programmes regroupés dans des modules.

Cahier des charges (CDC)

Définition

- ☐ **Document contractuel** entre :
 - un **client** (*maîtrise d'ouvrage, porteur du projet*)
 - son **prestataire** (*maîtrise d'œuvre, chef de projet*)
- ☐ Formalise les **besoins**, **contraintes** et **objectifs** devant être respectés lors de la conception du projet.
- ☐ Les **fonctionnalités** ainsi que le **résultat** attendu y sont définis.

Exemples simples

- ❑ **École** : l'énoncé d'un exercice ou d'un projet.
- ❑ **Société organisatrice de salons** : *Création d'une plateforme en ligne de vente et gestion de places pour des salons. Chaque visiteur pourra acheter ses tickets en ligne (le site devra être référencé), s'inscrire à des conférences et ateliers. Le staff devra pouvoir contrôler les tickets à l'entrée, placer les personnes sur leur siège, vérifier et pointer les participants à un atelier, [...]. Budget : 26000€, date butoire : 12 avril. L'interface devra être intuitive et s'utiliser sans besoin de formation.*
- ❑ **Fabricant/fournisseur** : *Création d'un support pour ordinateur portable réglable en hauteur (entre 2 et 30cm) et en inclinaison, léger, facile à ranger et à transporter. Le prix de fabrication ne devra pas excéder 5€. Les matériaux devront être solides et résistants au temps. Prévoir 3 coloris, utiliser au moins 43% de matériaux recyclés. Prévoir une housse de rangement.*

Étapes de développement d'un programme



Le rôle du développeur·se

L'objectif d'un·e développeur·se informatique est de **créer un logiciel**, une application ou un site web **répondant à un cahier des charges** plus ou moins complet.

Il/elle peut également jouer un rôle dans l'élaboration du cahier des charges.

- ❶ (facultatif) Participation à la rédaction du cahier des charges
- ❷ **Analyse du cahier des charges**
Extraction des informations utiles : méthode **DTI** (*Données, Traitements, Interfaces*), **ACD** (Analyse Chronologique Descendante).
- ❸ **Conception**
Création des algorithmes/logigrammes à partir des données et traitements identifiés, ainsi que des interfaces Homme/machine (IHM).
- ❹ **Implémentation**
Retranscription des algorithmes dans un langage de programmation. Choix des structures de données. Optimisations.
- ❺ **Tests** unitaires, d'**intégration**, puis de **validation (recette)**.
- ❻ **Livraison**

Analyse du CDC

Méthode Données Traitements Interfaces (DTI)

□ Données

Identification des données (noms) que l'on va traiter.

Ex : dans un logiciel de gestion des utilisateurs, on aura le nom, le prénom, la date de naissance (jour, mois, année)...

□ Traitements

Identification et organisation des traitements (verbes).

Ex : inscription d'un nouvel utilisateur.

□ Interfaces (Homme/machine – IHM)

Obtention des données (via le clavier, la souris...) et affichage des résultats (à l'écran).

Ex : formulaire de création d'un nouvel utilisateur avec 2 champs de texte (nom), 3 listes déroulantes (date), et un bouton (créer).

Analyse Chronologique Descendante (ACD)

Vise à identifier et ordonner de façon chronologique les traitements de notre application.

On part d'une description générale de ce que l'on veut faire (la tâche / le traitement à réaliser), puis on la décompose en plusieurs sous-parties, qu'on décompose elles aussi en sous-parties jusqu'à obtenir des sous-tâches très simples. L'objectif est notamment d'identifier les sous-tâches utilisées à plusieurs endroits.

Exemple :

Tâche : Inscription d'un utilisateur.

Décomposition : Récupération du prénom, récupération du nom, sauvegarde du nouvel utilisateur.

Décomposition de la sous-tâche « Récupération du prénom » :

Demander de saisir le prénom, récupérer le prénom au clavier (texte), etc.

Algorithmique

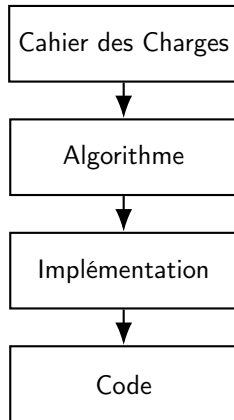
Définition

Suite définie et claire d'instructions permettant de résoudre un problème.

Les algorithmes doivent être compréhensibles par le plus grand nombre, sans nécessiter de connaissances dans un langage de programmation en particulier.

On les écrit donc dans une langue naturelle (e.g. *non informatique, le français ou l'anglais par exemple*).

On peut également les représenter sous la forme de schémas (ex. : *logigrammes*) ou de pseudo-code.



Exemple : Recette de cuisine

- ❑ **Données en entrée** : les ingrédients.
(*par ex. : salade, tomate, ail...*)
- ❑ **Algorithme** : instructions à effectuer dans l'ordre décrit. 
(mélanger, chauffer, découper, faire bouillir, laisser reposer X heures, répéter pour chaque X...)
- ❑ **Résultat** : un plat succulent, prêt à être mangé. 



Exemple : Notice IKEA

Certainement pour éviter de traduire toutes ses notices de montage, IKEA a fait le choix de schémas sans texte, les rendant par la même occasion, beaucoup plus claires. *Normalement...*

On voit bien : le résultat final, les prérequis et les instructions successives.

Crédits : Notice IKEA officielle





Exemple d'algorithme informatique (en français)

DÉBUT

ENTIER $n \leftarrow 0$

ÉCRIRE "Entrez un nombre :"

LIRE n

SI $n \geq 0$ ALORS

 ÉCRIRE $n + \text{" est positif ou nul."}$

SINON

 ÉCRIRE $n + \text{" est négatif."}$

FIN SI

FIN

⟨/⟩ Implémentation

En informatique, les algorithmes suivent des règles syntaxiques les rendant compréhensibles rapidement par les développeur.se.s. Ils sont généralement écrits en anglais.

Implémentation

Les algorithmes en informatique ont pour vocation d'être transposés dans un ou plusieurs langages de programmation.

On parle alors d'**implémentation** d'un algorithme.

Exemple : Implémentation en langage C de tel algorithme.

➤ Implémentation de l'algorithme précédent, en C

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int n = 0;
6
7     printf("Entrez un nombre :\n");
8     scanf("%d", &n);
9
10    if(n >= 0)
11    {
12        printf("%d est positif ou nul.", n);
13    }
14    else
15    {
16        printf("%d est negatif.", n);
17    }
18
19    return 0;
20 }
```

JS Implémentation de l'algorithme précédent, en JavaScript

```
1 var n = prompt("Entrez un nombre :");
2
3 if (n >= 0) {
4     console.log(n + " est positif ou nul.");
5 } else {
6     console.log(n + " est negatif.");
7 }
```

*Un même algorithme donne des codes différents une fois implémenté dans différents langages.
Mais il est compréhensible par tous, indépendamment de nos connaissances dans un langage.*

☰ Propriétés d'un algorithme



Calculabilité

Peut-on obtenir un résultat au problème en un nombre fini d'étapes ?



Complexité

Peut-on obtenir un résultat avec nos moyens et dans un temps raisonnable ?



Correction

Obtient-on toujours un/le résultat correct, quelles que soient les données en entrée ?



Réutilisabilité

Peut-on réutiliser notre algorithme pour des problèmes similaires ?



Calculabilité !

Existe-t-il un algorithme permettant, en un nombre fini d'étapes, de trouver un résultat à notre problème ?

Exemple :

On ne peut pas créer un algorithme qui retourne toutes les combinaisons possibles avec l'alphabet : *a, b, aa, ab, bb, aaa, aab, bonjour, ... , azofdhzjkjfodfohdzfoighdzfoz, ...*

- ✓ Ce problème est **non calculable**, car il n'a pas de limite claire.
- ✓ Pour qu'il soit **calculable**, il faut fixer une limite.
Exemple : *10 caractères maximum* $\Rightarrow$ calculable (mais **complexe**).

⚠ *Un algorithme ne peut résoudre un problème que s'il est défini avec des bornes claires.*

Complexité !

Estimation de la quantité de ressources
nécessaires à l'exécution d'un algorithme :

✓ Temps (🕒)

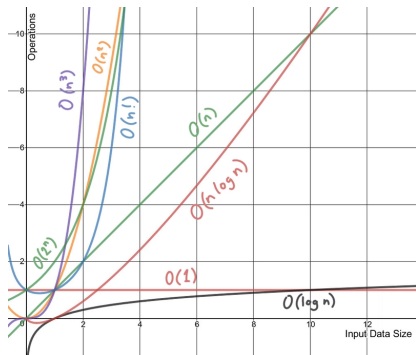
✓ Mémoire (💻)

En fonction de la quantité de données en
entrée, et donc du **nombre d'opérations à**
traiter.

Ressource supplémentaire :

Wikipédia – Analyse de la complexité des algorithmes

🔗 Schéma source





Complexité et sécurité !

La sécurité des mots de passe repose essentiellement sur la forte complexité des algorithmes de piratage (*Brute Force*) et la « lenteur » (😞) des ordinateurs lors de leur exécution.

D'après l'**Estimateur de temps pour cracker un mot de passe réalisé par BetterBuys**, un algorithme de *Brute Force* mettait environ :

- ✓ 10 ans en 2020 pour deviner ECE-Lyon
- ✓ (il en aurait mis le double 10 ans plus tôt)

Attention : Un mot de passe comme ECE-Lyon pourrait être trouvé :

- ✓ en quelques heures par un humain,
- ✓ en quelques secondes par dictionnaire !

⚠ Évitez de tester vos mots de passe personnels dans ces estimateurs.



Correction

Un algorithme est **correct** s'il produit toujours le résultat attendu, quelle que soit la variation des données en entrée.

Exemple :

Si notre algorithme est censé donner la somme de deux entiers qu'on lui fournit, alors en lui transmettant les entiers 3 et 5, il doit toujours donner le résultat 8, et uniquement celui-ci.



Réutilisabilité

Un algorithme est réutilisable s'il est suffisamment paramétrable en entrée pour pouvoir être utilisé dans plusieurs cas de figure.

Exemples :

- ✓ Une calculatrice dont les seuls boutons sont $[1+3]$ et $[=]$ est moins réutilisable qu'une calculatrice disposant de tous les chiffres ($[0]$, $[1]$, ..., $[9]$) et de tous les opérateurs ($[+]$, $[-]$, $[=]$, etc.).
- ✓ Un algorithme qui affiche « Bonjour Toto » est moins réutilisable qu'un algorithme demandant un prénom en entrée avant d'afficher un message de bienvenue. Le premier devrait être dupliqué pour dire bonjour à « Tata », par exemple.

Quand fait-on des algorithmes ?



Débutants : pour développer la logique informatique, sans pour autant avoir de connaissances dans un langage.



Professionnels : quand un problème est complexe et nécessite une réflexion approfondie. Le reste du temps, ils conçoivent mentalement l'algorithme en même temps qu'ils codent.



Experts / chercheurs : l'une de leurs missions peut consister à optimiser des algorithmes existants pour les rendre plus efficaces, ou à en concevoir de nouveaux. Ils peuvent choisir de les rendre publics ou non.



Vocabulaire : Algorithme naïf

Un algorithme rédigé instinctivement, sans recherche ni réflexion préalable, est appelé **algorithme naïf.**

D'autres algorithmes, ayant fait l'objet de recherches et d'optimisations, offrent de bien meilleurs résultats (en termes de complexité). Ils peuvent être trouvés sur le web, dans des livres spécialisés, etc.

Vocabulaire : Algorithme glouton / gourmand / goulu (*greedy algorithm*)

Un **algorithme glouton** est un algorithme qui suit le principe de faire, étape par étape, un choix optimal local, dans l'espoir d'obtenir un résultat optimum global.

Source : Wikipedia

◆ Comparaisons d'algorithmes de tri

Trier des données peut paraître simple, mais de nombreux algorithmes ont été inventés pour réaliser cette tâche. Ils sont plus ou moins efficaces selon la position initiale des données à trier.

 Play All	 Insertion	 Selection	 Bubble	 Shell	 Merge	 Heap	 Quick	 Quick3
Random								
Nearly Sorted								
Reversed								
Few Unique								

À voir : *Comparaison des principaux algorithmes de tri*

 Méthodologie de conception d'un algorithme**① Identifier les (potentielles) entrées**

De quelles données notre algorithme a-t-il besoin pour fonctionner ?

② Identifier les (potentielles) sorties

D'après les données reçues en entrée, quel résultat doit-il produire ?

③ Identifier et organiser les étapes / traitements

Quelles opérations effectuer et dans quel ordre pour obtenir le résultat ?

Les variables

Stockage des données

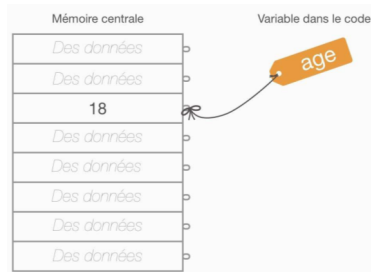
Les programmes informatiques traitent des données, enregistrées dans la mémoire temporaire de l'ordinateur (*mémoire centrale*).

Nous accédons à ces emplacements mémoire à l'aide de **variables**.

Une variable possède un **nom** et est rattachée à une donnée stockée en mémoire. Cette donnée peut évoluer au cours de l'exécution du programme.

Exemple : La variable `age` contient ici la valeur 18.

NB : deux variables ne peuvent pas avoir le même nom.





Les variables – Déclaration

Pourquoi déclarer une variable ?

Pour que l'ordinateur sache qu'il doit réserver un espace mémoire pour une variable dans notre programme, nous devons **déclarer cette variable**.

C'est lors de la déclaration que l'on donne à la variable son **nom**.

Celui-ci doit être unique, sauf cas particuliers que nous verrons plus tard.

NB : Le type d'une variable détermine la taille (en octets) qu'elle occupe en mémoire.



Les variables : $\leftarrow$ Affectation

Définition :

L'affectation de la valeur x à la variable nommée v consiste à stocker cette valeur dans l'espace mémoire associé à v :

variable (v) $\leftarrow$ valeur (x)

On dit que v prend la valeur x .

 Les variables : Initialisation**Définition :**

L'**initialisation** d'une variable correspond à sa **déclaration** accompagnée de sa **première affectation**.

Elle consiste à attribuer une **valeur initiale** à la variable dès le moment de sa déclaration.

Types de variables

Déclaration avec type :

- ✓ ENTIER age (déclare une variable entière nommée age)
- ✓ DÉCIMAL prix (variable à virgule)
- ✓ CARACTÈRE lettre

Selon les langages :

- ✓ En **C**, le type est obligatoire : `int age;`
- ✓ En **JavaScript**, le type est implicite : `let age = 18;`
- ✓ En algorithmique, on peut utiliser : `VARIABLE n`

 Instructions de base en algorithmique

Instruction	Rôle
DÉBUT / FIN	Encadre l'algorithme du début à la fin.
ENTIER age	Déclare une variable entière age (sans valeur initiale).
compteur $\leftarrow$ 2	Affecte la valeur 2 à compteur (doit être déclarée avant).
ENTIER compteur $\leftarrow$ 0	Déclare compteur et l'initialise à 0.
ENTIER annee, compteur $\leftarrow$ 3	Déclare deux variables : annee (non initialisée) et compteur (initialisée à 3).
VARIABLE prenom	Déclare une variable sans préciser le type (utilisé en algorithmique générique).

Les entrées/sorties

Instruction

Description

ÉCRIRE "Bonjour"

Affiche Bonjour dans la console.

ÉCRIRE "Le compteur indique
" + compteur

Affiche Le compteur indique 3 si la
variable compteur vaut 3.

LIRE age

Demande une valeur à l'utilisateur (saisie
clavier), et la stocke dans la variable age.



Les tests conditionnels

Structure simple (SI ... ALORS) :

SI *Condition* ALORS

Bloc d'instructions exécuté si la Condition est vraie

FIN SI

Structure complète (SI ... ALORS ... SINON) :

SI *Condition* ALORS

Bloc d'instructions exécuté si la Condition est vraie

SINON

Bloc d'instructions exécuté si la Condition est fausse

FIN SI

NB : Remplacez les parties en italique par ce qui est attendu. Un bloc d'instructions peut contenir plusieurs lignes.

Les tests conditionnels avancés

Structure avec SINON SI et SINON :

SI *Condition1* ALORS

Bloc1 exécuté si Condition1 est vraie

SINON SI *Condition2* ALORS

*Bloc2 exécuté si Condition1 est fausse
et Condition2 est vraie*

SINON

*Bloc3 exécuté si aucune des conditions
précédentes n'est vraie*

FIN SI

NB : On peut ajouter autant de SINON SI qu'on souhaite, mais un seul SINON, toujours à la fin. Une seule branche est exécutée : dès qu'une condition est vraie, son bloc est exécuté et les suivants sont ignorés.

Les boucles

Boucle POUR (itérative, bornée) :

```
POUR variable ← valeur1 À valeur2 PAR PAS DE incrément FAIRE  
    Bloc d'instructions répété  
FIN POUR
```

Le bloc est exécuté $(\text{valeur2} - \text{valeur1}) / \text{incrément} + 1$ fois.

Ex : valeur1 = 2, valeur2 = 5, incrément = 2 $\Rightarrow$ la boucle tourne avec variable = 2, puis 4.

Boucle RÉPÉTER n FOIS (répétition simple) :

```
RÉPÉTER n FOIS  
    Bloc d'instructions répété n fois  
FIN RÉPÉTER
```



Les boucles conditionnelles

Boucle TANT QUE (test en début) :

TANT QUE *Condition* FAIRE

Bloc d'instructions répété tant que la Condition est vraie

FIN TANT QUE

Boucle FAIRE ... TANT QUE (test en fin) :

FAIRE

Bloc d'instructions exécuté au moins une fois

TANT QUE *Condition*

La boucle FAIRE ... TANT QUE garantit l'exécution au moins une fois, même si la condition est fausse dès le début.

Indentation

L'**indentation** consiste à ajouter des espaces ou des tabulations en début de ligne pour décaler vers la droite le bloc d'instructions placé sous une structure de contrôle (SI, POUR, TANT QUE, etc.).

Exemple (non indenté vs indenté) :

☐ Non indenté

```
SI score > 1000 ALORS  
ÉCRIRE "Nouveau record."  
FIN SI
```

☒ Indenté

```
SI score > 1000 ALORS  
    ÉCRIRE "Nouveau record."  
FIN SI
```

L'indentation améliore la lisibilité et permet de savoir quelles instructions sont concernées par une condition ou une boucle.

Exemple d'algorithme — Parité d'un nombre

DÉBUT

ENTIER nombre $\leftarrow$ 0

ÉCRIRE "Saisissez un nombre au clavier :"

LIRE nombre

SI nombre % 2 = 0 ALORS

 ÉCRIRE "Ce nombre est pair."

SINON

 ÉCRIRE "Ce nombre est impair."

FIN SI

FIN

Logigramme

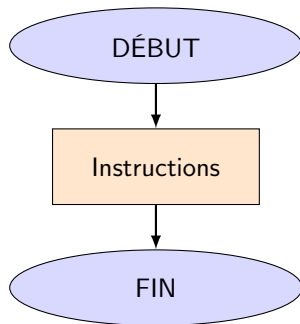


Algorithme graphique

Définition

Un **logigramme** est une représentation graphique normalisée de l'enchaînement des opérations d'un algorithme.

- ✓ Chaque instruction est placée entre les blocs **DÉBUT** et **FIN**.
- ✓ Les **flèches** indiquent la direction à suivre.
- ✓ Le chemin doit toujours mener à la **fin**.

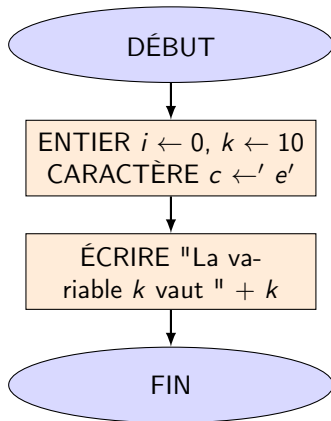


>_ Les instructions dans un logigramme

Rappel :

Les instructions se placent dans un **rectangle**, entre les blocs **DÉBUT** et **FIN**.

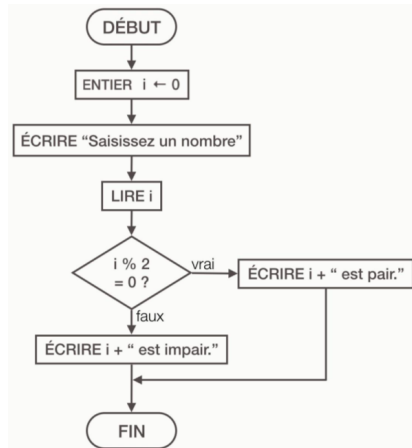
- ✓ Il est recommandé d'**initialiser toutes les variables**.
- ✓ Les instructions sont regroupées dans un seul bloc si elles sont consécutives.
- ✓ Chaque bloc ne contient qu'une seule action si c'est plus clair.



Tests conditionnels

Les **tests conditionnels** sont représentés par un **losange**.

- ✓ **Vrai** : la condition est vérifiée (flèche vers la sortie « vraie »).
- ✓ **Faux** : la condition n'est pas vérifiée (flèche vers la sortie « fausse »).
- ✓ Mieux vaut éviter les instructions et variables inutiles.



Exemple de test conditionnel avec deux branches (vrai/faux).

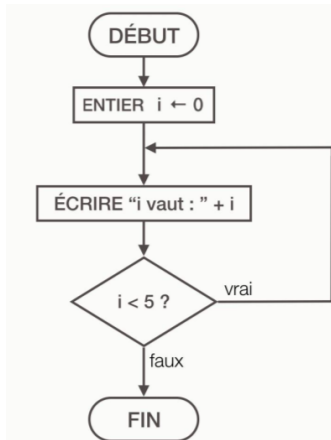
Les boucles : attention aux erreurs !

Remarque :

- ✓ Les boucles utilisent un test (comme les conditions), mais **reviennent en arrière**.
- ✓ Il faut **modifier une variable** pour ne pas avoir de boucle infinie.

Problème ici :

- ✓ La condition $i < 5$ est toujours vraie.
- ✓ La variable i n'est jamais modifiée $\Rightarrow$ **boucle infinie**.



Logigramme incorrect : pas d'incrémement de i

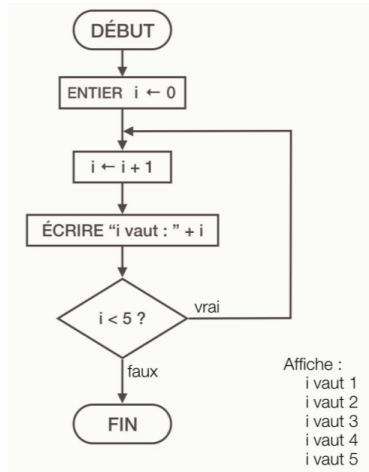
Les boucles - version corrigée : v1

Les boucles sont semblables aux tests, mais une des sorties revient en arrière.

- ✓ Ne pas oublier d'**incrémenter** la variable d'itération.
- ✓ Sinon, la condition reste vraie ⇒ **boucle infinie**.

Ici, la variable i est bien **modifiée à chaque passage**.

Affiche : i vaut 1, i vaut 2, ..., i vaut 5



Boucle fonctionnelle

Les boucles – version corrigée : v2

Les boucles sont semblables aux tests, à l'exception près qu'une des deux sorties revient en arrière.

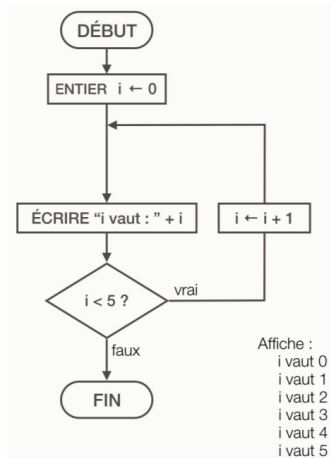
Déroulement :

- ✓ Initialisation : $i \leftarrow 0$
- ✓ Affichage, puis $i \leftarrow i + 1$
- ✓ Test : $i < 5$?
- ✓ Répétition tant que la condition est vraie

Résultat affiché :

- ✓ i vaut 0, 1, 2, 3, 4, 5

Remarque : l'incrémentation doit se faire au bon moment.



Boucle – Version 3

Les **boucles** sont semblables aux tests conditionnels, à la différence près qu'**une des deux sorties remonte** vers l'instruction précédente.

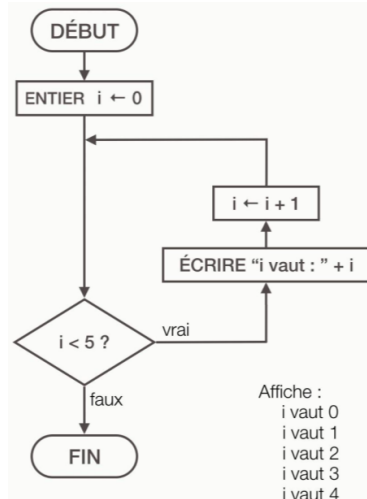
- ✓ Le test est placé dans un **losange**.
- ✓ Une **flèche remonte** vers le haut tant que la condition est vraie.
- ✓ Ici, i est **incrémentée avant** l'affichage.

Affichage obtenu : i vaut 1

i vaut 2

i vaut 3

i vaut 4



Exercices



Exercice/Exemple 1 : Nombre positif, négatif ou nul ?

Consigne :

Donnez un algorithme ainsi que le logigramme correspondant permettant de répondre au cahier des charges suivant :

Objectif

Nous souhaitons un programme qui :

- ✓ demande à l'utilisateur de saisir un nombre au clavier ;
- ✓ indique ensuite si ce nombre est **négatif**, **positif** ou **nul**.

✓ Correction - Exemple 1

DÉBUT

ENTIER $n \leftarrow 0$

ÉCRIRE "Saisissez un nombre"

LIRE n SI $n = 0$ ALORSÉCRIRE $n + \text{" est nul."}$ SINON SI $n > 0$ ALORSÉCRIRE $n + \text{" est positif."}$

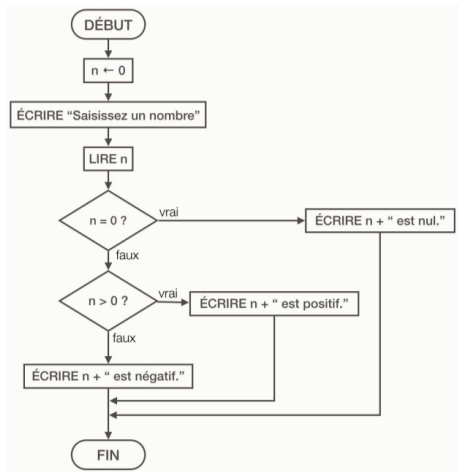
SINON

ÉCRIRE $n + \text{" est négatif."}$

FIN SI

FIN

NB : d'autres corrections sont possibles.





Exercice/Exemple 2 : Moyenne de nombres négatifs

Donnez un **algorithme** et son **logigramme** permettant à un utilisateur :

- ✓ de saisir autant de **valeurs négatives** qu'il le souhaite,
- ✓ puis d'en **afficher la moyenne**.

Les valeurs positives ne devront pas être prises en compte.

✓ Correction - Exemple 2

DEBUT

ENTIER somme $\leftarrow$ 0, nbElems $\leftarrow$ 0ENTIER newVal $\leftarrow$ -1, isFin $\leftarrow$ 0TANT QUE isFin $\neq$ 1 FAIRETANT QUE isFin $\geq$ 0 FAIRE

ÉCRIRE "Saisissez une valeur négative"

LIRE newVal

SI newVal $\geq$ 0 FAIRE

ÉCRIRE "Cette valeur n'est pas négative"

FIN SI

FIN TANT QUE

nbElems $\leftarrow$ nbElems + 1somme $\leftarrow$ somme + newValnewVal $\leftarrow$ 1

ÉCRIRE "Continuer à ajouter ? (oui : 0, non : 1)"

LIRE isFin

FIN TANT QUE

SI nbElems $\neq$ 0 FAIRE

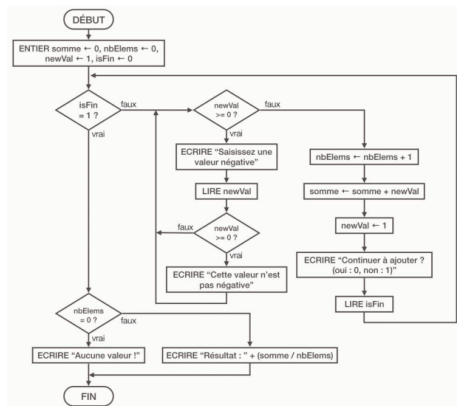
ÉCRIRE "Résultat : " + (somme / nbElems)

SINON

ÉCRIRE "Aucune valeur !"

FIN SI

FIN



✓ Mon algorithme est-il correct ?

Il est essentiel de **vérifier** que chaque algorithme conçu est correct.

- ✓ **Vérification** de l'adéquation du résultat avec le cahier des charges.
- ✓ **Jeux d'essais** : tester dans tous les cas de figure possibles.
 - Exemple : dans l'exercice précédent, tester le comportement avec un nombre négatif, positif et nul.
- ✓ **Preuves de terminaison** : s'assurer que l'algorithme se termine toujours (pas de boucle infinie) et fournit un résultat valide.
- ✓ **Traces d'exécution** : suivre étape par étape les valeurs prises par les variables.

NB : Ces vérifications sont à faire avant l'implémentation, mais peuvent aussi être réalisées après, sur le code.

Tester son programme



Tester son programme

Une fois l'algorithme implémenté, d'autres familles de tests entrent en jeu pour garantir la fiabilité du logiciel.

- ✓ Les tests permettent de détecter d'éventuels **effets de bord**, introduits par des modifications du code ou des ajouts de fonctionnalités.
- ✓ Ils évitent que des changements "cachent" ou "cassent" d'autres parties du programme.
- ✓ Un bon test peut révéler un dysfonctionnement même lorsque le code semble correct.

NB : Les tests sont souvent négligés, mais ils sont essentiels au bon fonctionnement d'une application sur le long terme.



Les tests unitaires

Les **tests unitaires** sont généralement écrits **avant** les portions de code qu'ils sont censés tester.

- ✓ Ils permettent de **vérifier le bon fonctionnement** d'une **partie précise** d'un logiciel (ex. : une fonction, un module...).
- ✓ Chaque test se concentre sur une **unité minimale de code**.

Exemple :

Vérifier que le module d'authentification renvoie bien un échec si le mot de passe fourni est faux.

Les tests d'intégration

Les **tests d'intégration** vérifient le bon fonctionnement de l'application **une fois tous les modules assemblés**.

- ✓ Ces modules doivent avoir été **déjà validés par les tests unitaires**.
- ✓ On teste leur **interaction** pour s'assurer que l'ensemble fonctionne correctement.



Les tests de validation

Les **tests de validation** visent à valider le **fonctionnement global de l'application** :

- ✓ **Respect du cahier des charges** et des exigences du client
- ✓ **Interface** utilisable et ergonomique
- ✓ **Performances et robustesse** :

Comment l'application réagit-elle si le nombre d'utilisateurs augmente considérablement ?